KR MANGALAM UNIVERSITY

School of Engineering & Technology

Course Code & Name

ENSP465 — New Age Programming Languages Lab

Capstone Project — Phase II

Implementation & Intermediate Demonstration Report

Project Title

Concurrent Web Scraper in Go

A goroutine-based worker-pool tool for fast, fault-tolerant extraction of data from many web pages

Field	Details
Student Name(s)	Kartikay Gautam, N.Yugesh
Roll Number(s)	2301010137,2301010199
Programme / Semester	B.Tech CSE — Semester VII
Academic Year	2026–27
Section	B.Tech CSE - C
Submission Date	10/11/2026
Faculty Name	Dr. Hemraj Kumawat School of Engineering & Technology, KRMU

1. Introduction and Phase II Status

1.1 Project Recap

The Concurrent Web Scraper is a command-line tool written in Go. It reads a list of web page URLs, downloads them in parallel with a pool of worker goroutines, extracts a fixed set of fields from every page (HTTP status, title, meta description, number of links, number of headings and response time), and writes the results to a JSON or CSV file together with a terminal summary. The problem, motivation and design were described in the Phase I report. This report covers Phase II: the implementation of that design and the first round of testing and measurement.

1.2 What Was Planned and What Is Done

Planned item (from Phase I)	Status	Notes
M1: Config and input loader	Done	Flags, validation, URL file loading, de-duplication
M2–M4: Dispatcher, worker pool, collector	Done	Goroutines, two channels, `sync.WaitGroup`, `context`
Fetcher with per-request timeout	Done	`net/http`, 10 s default, 5 MiB body cap
HTML parser	Done	`golang.org/x/net/html` tokenizer
M5: JSON output	Done	Ordered as in the input file
CSV output and terminal summary	Done	Ahead of the Phase I plan (was Phase III)
Retries with backoff	Done	Ahead of the Phase I plan (was Phase III)
Graceful shutdown (Ctrl+C)	Done	Partial results are saved
Unit tests, race check, benchmark	Done	17 test functions, 2 benchmarks
Per-host rate limiting	Not yet	Moved to Phase III (Section 7)

2. Development Environment and Project Structure

2.1 Environment

Item	Details
Language	Go 1.24 (the `go.mod` file requires Go 1.23 or newer)
External dependency	`golang.org/x/net` v0.38.0, used only for the `html` package
Everything else	Go standard library: `net/http`, `context`, `sync`, `encoding/json`, `encoding/csv`, `flag`, `os/signal`
Tooling	`go build`, `go test -race -cover`, `go vet`, `gofmt`

2.2 Project Structure

```
webscraper/
  cmd/scraper/main.go          command-line entry point
  cmd/benchmark/main.go        sequential vs concurrent benchmark
  internal/config/             flags, validation, URL loading   (M1)
  internal/model/              Job and PageResult types
  internal/fetcher/            HTTP download, error classification
  internal/parser/             HTML field extraction
  internal/scraper/            dispatcher, workers, collector   (M2-M4)
  internal/report/             JSON/CSV writers, summary        (M5)
  testdata/urls.txt            sample URL list
  README.md  go.mod
```

The program has 789 lines of Go in the main code and 474 lines of test code, split into two commands and six internal packages. Each package has one responsibility, and the packages map directly onto the modules M1 to M5 of the Phase I architecture.

3. Implementation Details

3.1 Configuration and Input Loading (M1)

The `config` package reads command-line flags with the standard `flag` package and validates every value before any work starts, so bad input fails fast with a clear message and exit code 2.

Flag	Default	Validation / meaning
`-input`	none	File with one URL per line (- reads standard input). URLs may also be given as arguments
`-workers`	10	Must be 1 to 100
`-timeout`	10s	Must be positive; applies to each HTTP request

Flag	Default	Validation / meaning
`-retries`	2	Must be 0 to 5
`-format`	json	`json` or `csv`
`-out`	results.<format>	Output file, or `-` for standard output

When loading the URL list, blank lines and lines starting with `#` are ignored, only `http` and `https` URLs with a host are accepted, duplicates are removed while keeping the original order, and rejected lines are reported as warnings instead of stopping the run.

3.2 Data Types

Two small types in the `model` package travel through the channels. A `Job` carries a URL and its original position. A `PageResult` is produced for every URL, including failed ones, so no URL is ever silently lost.

```
type Job struct {
    Index int      // position in the input, used to keep output ordered
    URL   string
}

type PageResult struct {
    Index        int    `json:"-"`
    URL          string `json:"url"`
    StatusCode   int    `json:"status"`
    Title        string `json:"title"`
    MetaDesc     string `json:"meta_description"`
    LinkCount    int    `json:"links"`
    HeadingCount int    `json:"headings"`
    ResponseMs   int64  `json:"response_ms"`
    Attempts     int    `json:"attempts"`
    Error        string `json:"error,omitempty"`
    ErrorKind    string `json:"error_kind,omitempty"`
}
```

3.3 The Concurrent Core (M2, M3, M4)

The `scraper` package implements the worker-pool pipeline shown in Figure 1. One dispatcher goroutine puts jobs into the `jobs` channel, N worker goroutines take jobs from it and put results into the `results` channel, and the main goroutine acts as the collector.

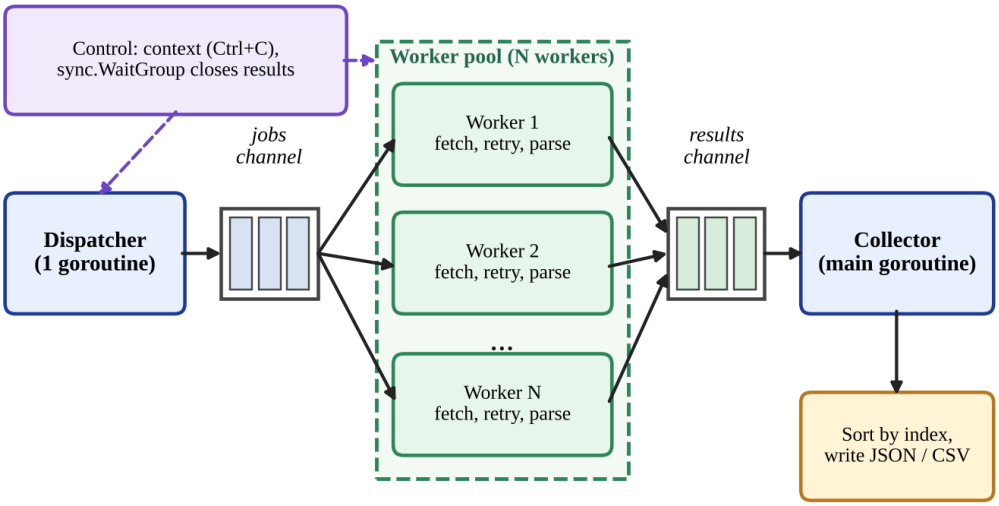


Figure 1: Goroutines and channels in the implemented scraper

The essential part of the `Run` function is shown below (comments shortened).

```
jobs := make(chan model.Job, workers)
results := make(chan model.PageResult, workers)

go func() {                                  // dispatcher
    defer close(jobs)
    for i, u := range urls {
        select {
        case jobs <- model.Job{Index: i, URL: u}:
        case <-ctx.Done():                   // Ctrl+C: stop handing out work
            return
        }
    }
}()

var wg sync.WaitGroup
for w := 0; w < workers; w++ {               // worker pool
    wg.Add(1)
    go func() {
        defer wg.Done()
        for job := range jobs {
            results <- s.process(ctx, job)
        }
    }()
}

go func() { wg.Wait(); close(results) }() // closer

for r := range results {                     // collector
    out = append(out, r)
}
sort.Slice(out, func(i, j int) bool { return out[i].Index < out[j].Index })
```

Several design choices keep this code simple and correct:

- **Only one goroutine closes each channel.** The dispatcher closes `jobs`, and a separate closer goroutine closes `results` after `wg.Wait()` shows that every worker has returned. This avoids sending on a closed channel.
- **A single collector, no mutex.** Only the collecting goroutine appends to the result slice, so no lock is needed and data races on the results cannot occur.
- **Dynamic load balancing.** Workers pull the next job when they are free, so a slow page never holds up the others.
- **Order is restored at the end.** Workers finish in an unpredictable order, so each result carries its `Index` and the collector sorts by it before output.
- **Clean cancellation.** If the context is cancelled, the dispatcher stops, the workers finish their current page and exit, and the run returns the results gathered so far.

A `RunSequential` function (same processing, no goroutines) was also written as the baseline for the benchmark.

3.4 Fetching and Error Classification

The `fetcher` package wraps `net/http`. Each request carries the run's context and the configured timeout, sets a User-Agent header, and reads at most 5 MiB of the response body so one huge page cannot exhaust memory. A response with status 400 or above is not treated as a transport error; the scraper decides what to do with it. Failures are sorted into categories that appear in the output and in the summary:

Error kind	Meaning	Retried?
`timeout`	Request exceeded the timeout or deadline	Yes
`network`	Connection refused, reset, or other transport failure	Yes
`http_5xx`	Server error (status 500 or above)	Yes
`http_4xx`	Client error such as 404 or 403	No (429 is retried)
`dns`	Host name could not be resolved	No
`parse`	Body could not be read as HTML	No
`cancelled`	Run was interrupted by the user	No

Permanent errors are not retried because a second attempt cannot succeed, and retrying them would only waste time and add load to the target server.

3.5 Retries with Backoff

The `process` function attempts each URL up to `retries + 1` times. After a temporary failure it waits 200 ms, then 400 ms, then 800 ms, doubling each time. The wait uses a `select` on the timer and the context, so pressing Ctrl+C interrupts a waiting worker immediately. The result records the number of attempts made.

3.6 HTML Parsing

The `parser` package uses the streaming tokenizer from `golang.org/x/net/html`, which reads the page once without building a tree. It records the first `<title>` (with whitespace collapsed and HTML entities decoded), the `content` of the `<meta name="description">` tag (case-insensitive), the number of `<a>` tags that have a non-empty `href`, and the number of heading tags `<h1>` to `<h6>`. Text inside `<script>` is treated by the tokenizer as plain text, so fake tags inside scripts are not counted. Malformed HTML does not cause errors.

3.7 Reporting

The `report` package writes the results as an indented JSON array (the `error` fields are omitted for successful pages) or as CSV with a header row, with commas and quotes inside values escaped correctly. It also computes the run summary: URLs supplied, succeeded, failed (broken down by error kind), skipped, average response time of successful pages and total wall-clock time. The total time is measured by a stopwatch around the whole `Run` call, so it is the real elapsed time, not the sum of the page times.

3.8 Graceful Shutdown

The command creates its context with `signal.NotifyContext` for Ctrl+C and termination signals. On interruption the pool stops taking new work, the partial results are still written to the output file, and the program exits with code 130 after printing a note.

4. Testing and Results

4.1 Test Strategy

Package	What the tests verify
`parser`	Ten table-driven cases: full page, empty input, missing tags, anchors without `href`, only the first title, case-insensitive meta, all heading levels, tags inside scripts, malformed HTML, HTML entities

Package	What the tests verify
`config`	URL loading (comments, blanks, duplicates, invalid and non-HTTP lines, whitespace) and flag validation (missing input, worker, retry, timeout and format limits, unknown flags)
`fetcher`	User-Agent header, non-2xx responses returned without error, 5 MiB body cap, malformed URL, error classification
`scraper`	Output order and extracted values for 30 pages; mixed 404, 500 and refused-connection failures with the right retry counts; timeout classification; a fake flaky fetcher that succeeds on the third attempt; cancellation in the middle of a run; empty input; a speed-up check of concurrent versus sequential
`report`	Valid JSON, empty list encoded as `[]`, valid CSV with commas and quotes, summary counts and text

The tests use `net/http/httptest` to start real local HTTP servers, so no internet access is needed and the results are repeatable. Because the scraper depends on the small `Fetcher` interface, a fake fetcher is used to test the retry logic without any network.

4.2 Test Results

All 17 test functions pass with the Go race detector enabled (`go test -race -cover ./...`), which reported no data races. Statement coverage by package:

Package	Coverage	Comment
`internal/parser`	97.7%	
`internal/config`	95.7%	
`internal/report`	95.1%	
`internal/scraper`	89.0%	Core concurrent logic
`internal/fetcher`	88.5%	
`cmd/scraper`, `cmd/benchmark`, `internal/model`	0%	Entry points tested manually; `model` holds only type definitions

4.3 Benchmark: Sequential versus Concurrent

The `cmd/benchmark` program starts a local HTTP server that waits 200 ms before answering each request, to simulate network latency, and then scrapes 100 pages sequentially and with

different numbers of workers. Using a local server makes the measurement repeatable and shows the effect of concurrency alone.

Mode	Time	Speed-up	Pages OK
Sequential (baseline)	20.09 s	1.0×	100/100
2 workers	10.05 s	2.0×	100/100
5 workers	4.02 s	5.0×	100/100
10 workers	2.01 s	10.0×	100/100
20 workers	1.01 s	19.9×	100/100
50 workers	0.42 s	47.6×	100/100
100 workers	0.22 s	92.9×	100/100

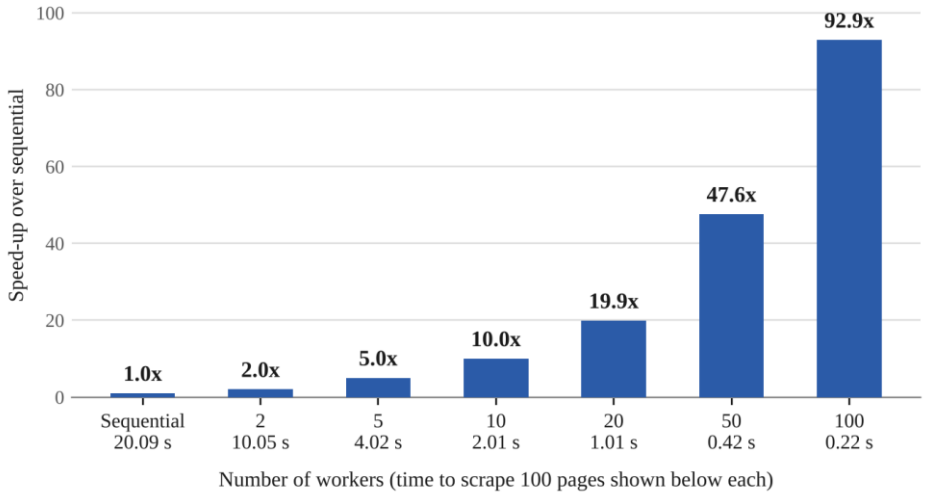


Figure 2: Speed-up over sequential scraping (100 pages, 200 ms server delay)

The speed-up is almost exactly equal to the number of workers, because each request spends nearly all of its time waiting. The Go benchmarks in `scraper_test.go` agree: scraping 50 pages with a 20 ms delay took about 1.04 s sequentially and 0.11 s with 10 workers. These numbers come from a local server and measure the concurrency design; times on real websites depend on the network and on each site.

4.4 Run on Real Websites

The tool was also run on a list of 24 real URLs: pages from scraping-practice sites, general documentation and reference sites, and a few deliberately failing addresses (a missing page, a server error and a non-existent domain) to exercise error handling. The terminal output is shown in Figure 3.

```
---------------- Summary ----------------
URLs supplied : 24
Succeeded     : 20
Failed        : 4
  - dns       : 1
  - http_4xx  : 2
  - http_5xx  : 1
Avg response  : 868 ms (successful pages)
Total time    : 3.126s
Results written to results.json
```

Figure 3: Summary printed by the scraper for the 24-URL run

Measure	Value
URLs supplied	24
Succeeded / failed	20 / 4
Failures by kind	dns: 1, http_4xx: 2, http_5xx: 1
Average response time (successful pages)	868 ms
Total time	3.126 s

The 20 successful pages took about 868 ms each on average, roughly 17 s of cumulative waiting. The whole run finished in about 3.1 s because the pages were downloaded in parallel, which is of the order of five times faster than fetching them one by one (this is an estimate from the averages). The four failures were reported by category, and none of them stopped the run.

4.5 Sample Output

A test run against a small local website produced the following record in the JSON file (one entry per URL, in input order). The CSV file contains the same fields as columns, plus `error_kind` and `error`, which are filled only for failed pages.

```
{
  "url": "http://127.0.0.1:8765/index.html",
  "status": 200,
  "title": "Demo Home",
  "meta_description": "Home page of the demo site",
  "links": 2,
  "headings": 1,
  "response_ms": 21,
  "attempts": 1
}
```

5. Comparison with the Phase I Objectives

No.	Objective (Phase I)	Status	Evidence
1	Worker pool with 1 to 100 goroutines, configurable	Achieved	`-workers` validated to 1–100; benchmark run with 1 to 100 workers
2	At least 5× speed-up for 100 pages with 10 workers (200 ms server delay)	Achieved	10.0× (2.01 s against 20.09 s)
3	Extract six fields plus response time	Achieved	URL, status, title, meta description, links, headings, response time; verified in tests
4	Fault tolerance: 10 s timeout, 2 retries, failures recorded	Achieved	Defaults as specified; tests for 404, 500, refused connection, timeout and recovery on retry
5	JSON and CSV output, 70% coverage, race-free	Achieved	Both formats and summary implemented; 88–98% coverage in core packages; `-race` clean

One in-scope item from Phase I is not complete yet: per-host rate limiting. It is planned for Phase III.

6. Design Issues and How They Were Handled

Issue	Solution
Workers finish in a different order from the input, but the output should match the input	Each job and result carries its original `Index`; the collector sorts by it before writing
Retrying a 404 is pointless and adds load to servers	Errors are classified; only timeouts, network errors, 5xx and 429 are retried
Risk of deadlock or leaked goroutines on cancellation	The dispatcher uses `select` on the context; workers always deliver their result; the collector reads until `results` is closed
Shared results slice could cause data races	Only the collector goroutine writes to it, so no mutex is needed; confirmed with `-race`
A very large page could use too much memory	The response body is limited to 5 MiB

Issue	Solution
Retry logic is hard to test with a real network	The `Fetcher` interface lets tests use a fake that fails a set number of times
Waiting between retries should not delay Ctrl+C	The backoff wait is a `select` between a timer and the context

7. Limitations and Phase III Plan

7.1 Current Limitations

- There is no per-host rate limiting yet, so a high worker count aimed at one website could overload it. Mixing many different sites avoids this for now.
- `robots.txt` is not parsed [10], and JavaScript-generated content is not executed. Both were out of scope in Phase I.
- The retry backoff has no random jitter.
- The command-line entry point is tested by hand, not by automated tests.

7.2 Plan for Phase III

No.	Tentative dates	Work
1	By 18 Nov 2026	Add per-host rate limiting with `golang.org/x/time/rate` and tests for it
2	19 – 26 Nov 2026	Automated tests for the command-line layer; measured sequential and concurrent runs on real sites; final race-detector and coverage runs
3	27 Nov – 4 Dec 2026	README and final report, code cleanup, demonstration and viva preparation

8. How to Build, Run and Demonstrate

```
go mod tidy                                  # download dependencies
go build -o scraper ./cmd/scraper            # compile
go test -race -cover ./...                   # tests, race detector, coverage
go run ./cmd/benchmark                       # sequential vs concurrent table

./scraper -input testdata/urls.txt           # scrape, write results.json
./scraper -input testdata/urls.txt -workers 1    # one at a time
./scraper -input testdata/urls.txt -workers 10   # ten at a time
./scraper -input testdata/urls.txt -format csv -out results.csv
```

The demonstration compares the `Total time` line of the `-workers 1` and `-workers 10` runs, shows the failure categories in the summary, and opens the JSON and CSV outputs.

9. References

[1] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[2] The Go Authors, "Package context," Go Packages. [Online]. Available: https://pkg.go.dev/context. Accessed: Nov. 2026.

[3] The Go Authors, "Package net/http," Go Packages. [Online]. Available: https://pkg.go.dev/net/http. Accessed: Nov. 2026.

[4] S. Ajmani, "Go Concurrency Patterns: Pipelines and cancellation," The Go Blog, Mar. 2014. [Online]. Available: https://go.dev/blog/pipelines. Accessed: Nov. 2026.

[5] The Go Authors, "Package golang.org/x/net/html," Go Packages. [Online]. Available: https://pkg.go.dev/golang.org/x/net/html. Accessed: Nov. 2026.

[6] The Go Authors, "Data Race Detector," go.dev. [Online]. Available: https://go.dev/doc/articles/race_detector. Accessed: Nov. 2026.

[7] The Go Authors, "Package testing," Go Packages. [Online]. Available: https://pkg.go.dev/testing. Accessed: Nov. 2026.

[8] The Go Authors, "Package net/http/httptest," Go Packages. [Online]. Available: https://pkg.go.dev/net/http/httptest. Accessed: Nov. 2026.

[9] The Go Authors, "Package os/signal," Go Packages. [Online]. Available: https://pkg.go.dev/os/signal. Accessed: Nov. 2026.

[10] M. Koster, G. Illyes, H. Zeller, and L. Sassman, "Robots Exclusion Protocol," IETF, RFC 9309, Sep. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9309. Accessed: Nov. 2026.